LinkiaFP

PLANTILLA: Entrega de Actividades

6-27_1_1_M-IA_5074_Sistemas de Big Data

MASTER IA & BIG DATA

Curso académico 2026/2027 - Edición Online

En **FUCSIA** los textos de los enunciados.
En **NEGRO** las respuestas.

Acceso al repositorio de ejercicios para el Módulo actual sobre Sistemas de Big Data
https://github.com/dcueli-com/bigdata_sys.git

Tema 1: Fundamentos de Análisis de Datos y Técnicas Básicas: Dosier de Actividades Obligatorias Archivo

EJERCICIO 4: Utiliza la librería SpaCy de Python para cargar un modelo de lenguaje y procesar un texto en busca de nombres propios.
******Como parte opcional, podéis hacer pruebas con textos propios o usar textos existentes para probar se eficiencia.******

Acceso al recurso del repositorio para este ejercicio:

- Notebook
  https://github.com/dcueli-com/bigdata_sys/blob/develop/src/activities/RA1/lesson_01/notebooks/1_4_spacy.ipynb
- Python simple script
  https://github.com/dcueli-com/bigdata_sys/blob/develop/src/activities/RA1/lesson_01/scripts/unified.py

Como me gusta ponerme las cosas fáciles, como texto de ejemplo he seleccionado uno de tipo *Ipsum*, pero para darle un toque más diferenciador, por así decirlo, me he decantado por el *Chiquito Ipsum*, en vez de el tan trillado y aburrido *Lorem Ipsum*.

> *"Lorem fistrum se calle ustée pupita occaecat pecador pupita enim consequat condemnor diodeno. Aute ut aliquia diodeno. A peich fistro hasta luego Lucas sit amet nisi officia está la cosa muy malar commodo torpedo atomatoa que dise usteer. No puedor pecador papaar papaar no puedor no te digo trigo por no llamarte Rodrigor. La caidita diodeno occaecat ahorrar enim a peich. Irure a gramenawer mamaar caballo blanco caballo negroll. Magna fistro velit pupita cillum mamaar no te digo trigo por no llamarte Rodrigor llevame al sircoo no te digo trigo por no llamarte Rodrigor la caidita magna. A gramenawer occaecat magna occaecat officia diodeno no puedor. Caballo blanco caballo negroll va ustée muy cargado a wan minim sit amet voluptate.*
>
> *Labore fistro al ataqueerr! quis está la cosa muy malar aliquip ut. Et está la cosa muy malar velit magna cillum torpedo enim pecador. A wan commodo reprehenderit apetecan ullamco qui ese pedazo de va usté muy cargado sit amet. Dolor tiene musho peligro que dise ustéee apetecan consectetur no puedor reprehenderit. Hasta luego Lucas par la gloria de mi madre ut diodeno te va a hasé pupita hasta luego Lucas quietooor. Velit qui irure cillum eiusmod ex maaar dolore occaecat."*

Justificación de la Arquitectura y Preprocesamiento
Para la extracción de nombres propios (NER), como indica el enunciado, he empleado dos modelos de SpaCy con enfoques complementarios ya que, como sabemos, los textos aleatoriamente generados Lorem Ipsum están escritos en Latín y, a sabiendas también del Español, que el maestro Chiquito se inventaba las palabras, por todo esto se entiende que puedan ser textos con relativa complejidad o mixtos.
Los modelos seleccionados son:

- **es_core_news_sm,** entrenado para el idioma Español.
- ***la_core_web_md***, modelo entrenado para el idioma Latín.

Normalización de Esquemas de Etiquetado
Durante la ejecución e inspección de los resultados de los modelos he identificado un aspecto importante para el etiquetado. Ambos modelos usan etiquetados distintos para la identificación de nombres propios ya que los utilizados para poder operar apropiadamente con dichos modelos:

- En *Español* (*es_core_news_sm*) utiliza la etiqueta **PER** para identificar personas.
- El modelo de *Latín* (*la_core_web_md*), al parecer, después de usar, revisar y buscar, implementa el estándar OntoNotes, utilizando la etiqueta **PERSON**.

Evaluación Individual de Modelos
Español (es_core_news_sm)
Aunque tenga un alto rendimiento para capturar vocabulario y estructuras gramaticales modernas con respecto al texto seleccionado presenta las evidentes **limitaciones**, como puede ver primeramente dando una mayor tasa de falsos positivos en palabras con la mayúscula inicial, véanse sustantivos o palabras de origen no castellano debido a la inclusión del Latín.

```
+ ============================================================
+ LANGUAGE MODEL: -Spanish-
+ Persons proper names extracted from the text (PER label):
+ ------------------------------------------------------------
+
Proper name: Lucas
Proper name: Rodrigor
Proper name: Rodrigor
Proper name: Caballo
Proper name: wan minim
Proper name: amet voluptate
Proper name: Labore
Proper name: Dolor
Proper name: Lucas
Proper name: Lucas
+
+ All detected entities (for comparison):
+ ------------------------------------
Text: Lorem                         | Type/Label: MISC
Text: occaecat pecador              | Type/Label: MISC
Text: Aute ut aliquia diodeno       | Type/Label: MISC
Text: Lucas                         | Type/Label: PER
Text: No puedor pecador papaar      | Type/Label: MISC
Text: Rodrigor                      | Type/Label: ORG
Text: occaecat                      | Type/Label: LOC
Text: peich                         | Type/Label: LOC
Text: Irure                         | Type/Label: ORG
Text: Magna                         | Type/Label: MISC
Text: Rodrigor                      | Type/Label: PER
Text: Rodrigor                      | Type/Label: PER
Text: A gramenawer occaecat         | Type/Label: MISC
Text: occaecat                      | Type/Label: LOC
Text: Caballo                       | Type/Label: PER
Text: wan minim                     | Type/Label: PER
Text: amet voluptate                | Type/Label: PER
Text: Labore                        | Type/Label: PER
```

```
Text: Et está la cosa muy malar   | Type/Label: MISC
Text: A wan commodo reprehenderit | Type/Label: MISC
Text: Dolor                       | Type/Label: PER
Text: Lucas                       | Type/Label: PER
Text: hasé                        | Type/Label: LOC
Text: Lucas                       | Type/Label: PER
Text: Velit qui irure             | Type/Label: MISC
Text: occaecat                    | Type/Label: LOC
+
+ Time to load: 0.0367 seconds
+ ============================================================
```

Latín (la_core_web_md)
Para las palabras en Latín, como comenté anteriormente, hemos añadido el modelo ***la_core_web_md***. Se evidencia claramente que tiene la sensibilidad a los términos de origen Latín y declinados ya que tiene menos capturas de términos, pero al ser el texto como es, puede omitir nombres o expresiones propias del español.

```
+ ============================================================
+ LANGUAGE MODEL: -Latin-
+ Persons proper names extracted from the text (PER label):
+ ------------------------------------------------------------
+
Proper name: Lucas
Proper name: Rodrigor
Proper name: Magna
Proper name: Rodrigor
Proper name: Rodrigor
Proper name: Caballo
Proper name: Dolor
Proper name: Hasta
Proper name: Lucas

Proper name: Lucas
+
+ All detected entities (for comparison):
+ ------------------------------------
Text: Lucas                     | Type/Label: PERSON
Text: Rodrigor                  | Type/Label: PERSON
Text: La                        | Type/Label: NORP
Text: Irure                     | Type/Label: LOC
Text: Magna                     | Type/Label: PERSON
Text: Rodrigor                  | Type/Label: PERSON
Text: Rodrigor                  | Type/Label: PERSON
Text: Caballo                   | Type/Label: PERSON
Text: Dolor                     | Type/Label: PERSON
Text: Hasta                     | Type/Label: PERSON
Text: Lucas                     | Type/Label: PERSON
Text: Lucas                     | Type/Label: PERSON
Text: Uelit                     | Type/Label: NORP
+ Time to load: 0.0899 seconds
+ ============================================================
```

Estrategia de la intersección híbrido
Para maximizar la precisión y filtrar el ruido de falsos positivos, he implementado la intersección de los dos modelos.

Mecanismo de Funcionamiento

1. **Extracción de Candidatos**
   Genero dos conjuntos independientes con las entidades identificadas por “nombre propio” para cada modelo por su etiqueta.
2. **Cálculo de la Intersección**
   La función `intersection()` evalúa el resultado anterior y conserva **únicamente los nombres propios confirmados por ambos modelos simultáneamente**.
3. **Filtrado de Falsos Positivos**
   Si el modelo Español marca erróneamente un sustantivo por mayúscula puntual pero el modelo de latín no lo reconoce como entidad de persona, el término es descartado.
4. **Métricas y Resultados**
   La intersección reduce drásticamente los falsos positivos, devolviendo un conjunto de entidades de alta confiabilidad, pero también cabe reseñar que pueden ocurrir a la vez, falsos negativos

```
+ ================================================
+ HYBRID or INTERSECTION (Spanish  &  Latin MODELS)
+ Confirmed proper names (Detected by BOTH models):
+ ------------------------------------------------
+
Confirmed Name: Dolor
Confirmed Name: Lucas
Confirmed Name: Caballo
Confirmed Name: Rodrigor
+
+ Summary metrics:
+ ------------------------------------------------
Spanish model candidate count : 7
Latin model candidate count   : 6
Consensus candidates count    : 4
+
+ Time to evaluate intersection: 0.0020 seconds
+ ================================================
```

Conclusión
El uso de un único modelo preentrenado puede resultar insuficiente en textos ambiguos o de dominio específico como es en este caso que se pretende analizar textos híbridos entre Latín y Español.

Creo que la combinación de modelos mediante **intersección** puede parecer una estrategia eficaz para mitigar la desventaja de cada modelo individual al enfrentarse al tipo de texto elegido, y así se incrementa la precisión en la extracción sin necesidad de reentrenar un modelo desde cero.

EJERCICIO 5: Desarrolla un script en Python que realice web scraping de una tienda de libros en línea.
En este ejercicio, se pide crear un programa que extraiga información sobre libros de la página web "http://books.toscrape.com/".

- ***Utiliza las bibliotecas requests y BeautifulSoup para realizar web scraping de la página principal de "http://books.toscrape.com/".***
- ***Extrae el título y el precio de cada libro listado en la página.***

Para realizarlo tendrás que mirar bien que contiene el Html de la web para saber que contenido extraer.

Acceso al recurso del repositorio para este ejercicio:
https://github.com/dcueli-com/bigdata_sys/blob/develop/src/activities/RA1/lesson_01/scripts/1_5_scrapper.py

Bueno como me sigue gustando ponérmelo fácil, vamos a explicar qué he hecho aquí.

Planteamiento y Metodología
Mi solución a este ejercicio ha sido, como hasta ahora, usar el mismo proyecto pero con apartados diferentes. He diseñado un script modular en Python utilizando las librerías que se han pedido como requisito

- ***requests***, para las peticiones HTTP y
- ***BeautifulSoup***, para la navegación e inspección del árbol DOM.

Aquí ya me he decidido, que es a lo que estoy acostumbrado al “divide y vencerás” en contra de algo monolítico, por un empaquetado editable (pip install -e .), aislando configuraciones como por ejemplo, definiciones de constantes en un archivo separado src/config/globals.py y herramientas o utilidades de ayuda como son funciones para acciones repetitivas en src/utils/helpers.py.

Análisis del HTML e Inspección de Elementos
Tras inspeccionar el código fuente de la página objetivo de este ejercicio, Books to Scrape, claramente se puede ver y acceder a los selectores clave para la captura de datos:

- El **Contenedor principal** de cada libro está estructurado dentro de una etiqueta `<article>` con la clase “`class=product_pod`”.
- Para el **Título del libro**, el texto visible del enlace dentro del `<h3>` se encuentra truncado visualmente para títulos que se exceden en longitud, así que me he decido por extraer el atributo “`title`” de la etiqueta por contener la cadena de texto completa.
- El **Precio** se localiza directamente dentro de un elemento `<p>` con la clase específica “`class=price_color`”.

Ver captura de pantalla adjunta

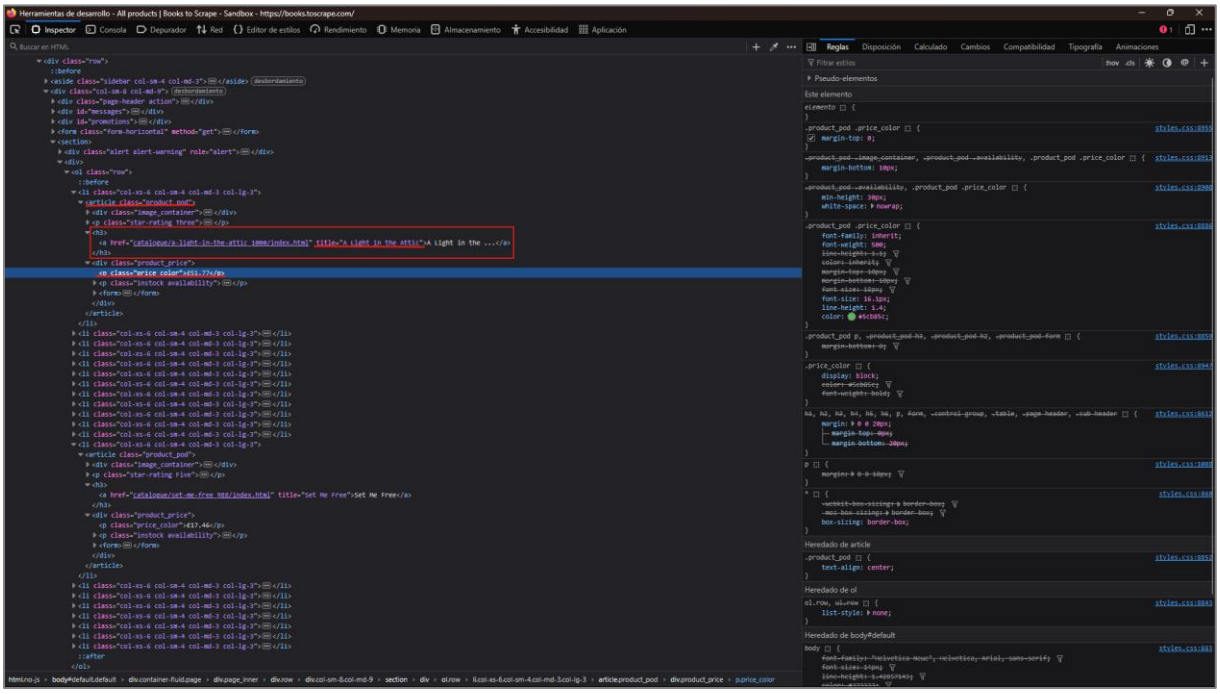


Fig. 1. Ventana desarrollador en Firefox

Flujo de Ejecución del Script

1. **Petición HTTP**
   Se realiza el `requests.get()` a la URL objetivo (constante `TARGET_URL`), validando que el estado de la respuesta sea exitoso (`200 OK`).
2. **Analizando del DOM**
   Se analiza (parse) el contenido HTML y se observa que el contenedor principal para cada libro es la etiqueta `<article>` con clase "`class=product_pod`".
3. **Extracción y Limpieza**: Se recorre cada elemento en la iteración `.find_all` extrayendo en las variables `bookTitle` (el atributo `title`) y `bookPrice` (texto limpio del párrafo), gestionando posibles ausencias de etiquetas.
4. **Salida Formateada**: Se presentan los resultados por consola aplicando formatos de alineación para facilitar la lectura de los títulos y precios extraídos.

EJERCICIO 6: Crear una base de datos SQL con información de libros y realizar varias consultas para analizar los datos utilizando el compilador SQL en línea de Programiz. En este ejercicio, utilizarás los datos de libros obtenidos del ejercicio anterior de web scraping. Crearás una base de datos, insertarás los datos y realizarás diversas consultas SQL para extraer información útil.

Pasos a seguir:

1. ***Acceso al Compilador SQL en línea:***
   - ***Visita https://www.programiz.com/sql/online-compiler/***
2. ***Creación de la Base de Datos:***
   - ***Crea una tabla llamada 'books' con las siguientes columnas:***
   - ***id (INTEGER, clave primaria, autoincremento)***
   - ***title (TEXT)***
   - ***price (DECIMAL(10,2))***

3. *Inserción de Datos:*
   - *Inserta al menos 10 libros en la tabla 'books' utilizando los datos obtenidos del web scraping (si no lo has conseguido puedes poner propios).*
4. *Consultas SQL:*
   Realiza las siguientes consultas:
   - *Selecciona todos los libros y muéstralos ordenados por precio de forma descendente*
   - *Calcula el precio promedio de todos los libros.*
   - *Encuentra el libro más caro y el más barato.*
   - *Cuenta cuántos libros tienen un precio superior a $50.*
   - *Selecciona los títulos de los 5 libros más baratos.*
   - *Calcula el precio total de todos los libros en la base de datos.*
5. *Consulta Avanzada:*
   - *Crea una consulta que muestre el título del libro, su precio, y una columna adicional que indique si el libro es "Barato" (menor al promedio) o "Caro" (mayor o igual al promedio).*

Como soy de la vieja escuela (y me gusta ponerme las cosas fáciles), podía haber contenerizado en Docker, Kubernetes, etc; pero como ya tenía instalado el motor de MySQL localmente, he preferido hacerlo ahí, aunque lo haré igualmente en programiz.com, así que voy a detallar con capturas de pantalla lo que se pide en el enunciado.

1. ***Acceso al compilador***
2. ***Creación de la Base de Datos***
   Consultas DML y DDL donde creamos el Esquema (*maibd_bigdata_sys*) y la BD (*ra_1_5_books*).
   Debo añadir que este esquema lo utilizaré más adelante para el resto de ejercicios que se planteen.
3. ***Inserción de Datos***
   Adición de la información a dicha tabla con el contenido extraído del ejercicio anterior.

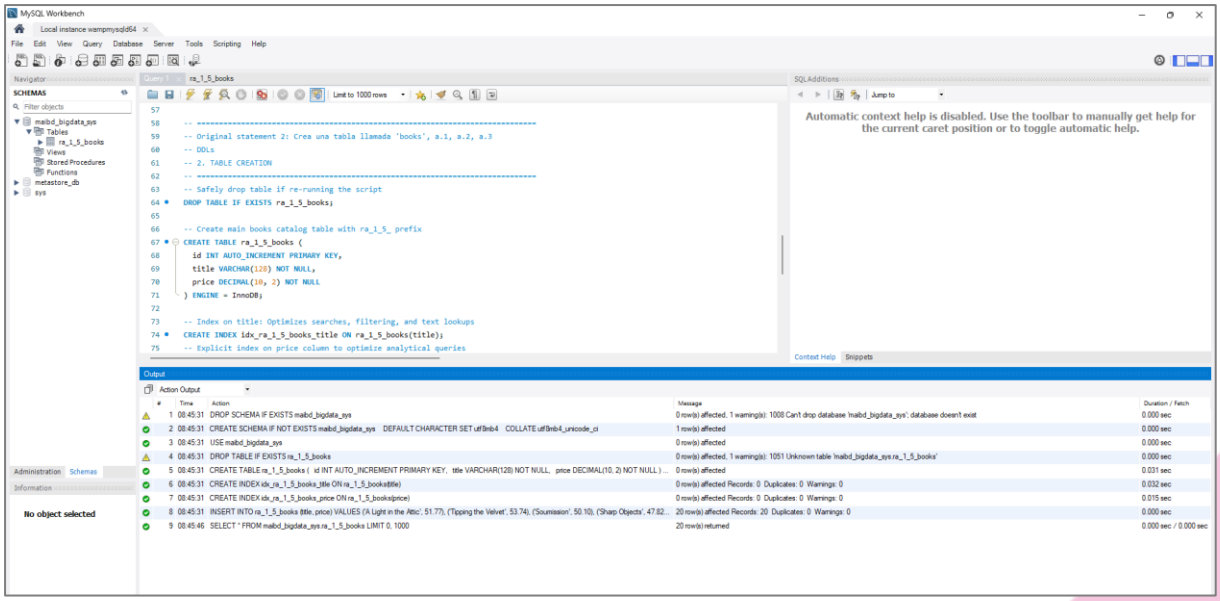


Fig. 2. MySQL Workbench



Fig. 3. programiz.com, *mencionar que aquí no es posible crear un Esquema nuevo y mucho menos indicar el motor de almacenamiento como en la consulta específica en MySQL, por lo que he optado por usar SQL estándar (ANSI SQL) y nos quitamos de problemas.*

4. Consultas SQL

- *Realiza las siguientes consultas:*
  a. *Selecciona todos los libros y muéstralos ordenados por precio de forma descendente*

```sql
SELECT
  id,
  title,
  price
FROM
  ra_1_5_books
ORDER BY
  price
DESC;
```

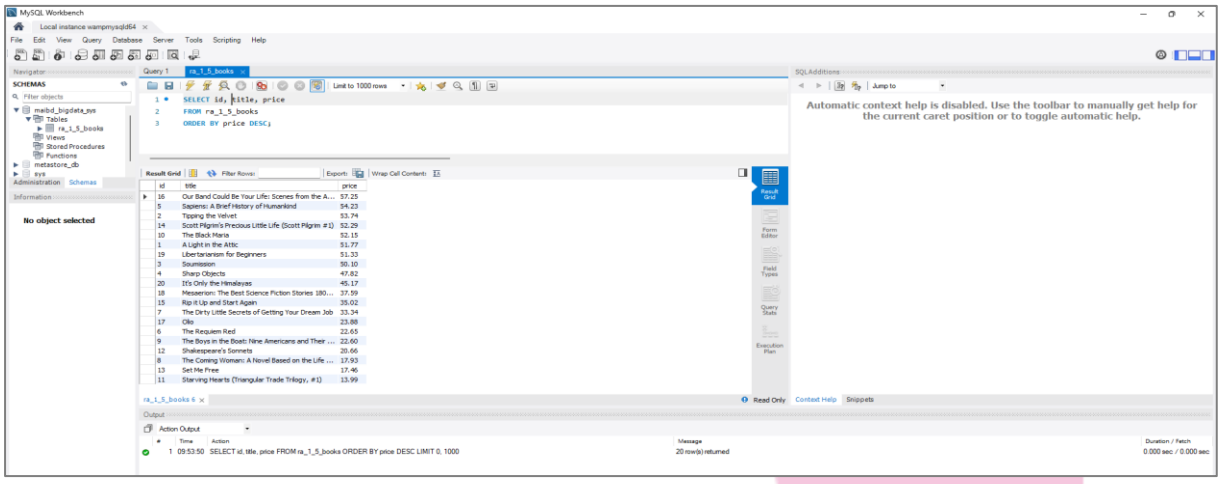


Fig. 4. MySQL Workbench

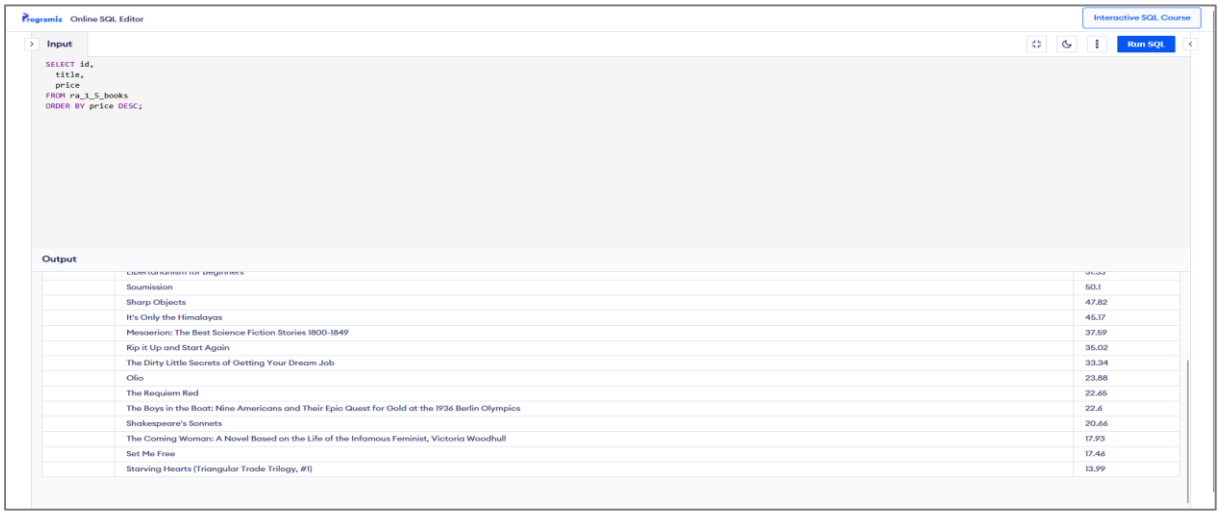


Fig. 5. Programiz

b. Calcula el precio promedio de todos los libros.

```sql
SELECT
  ROUND(AVG(Price), 2) AS average_price
FROM
  ra_1_5_books;
```

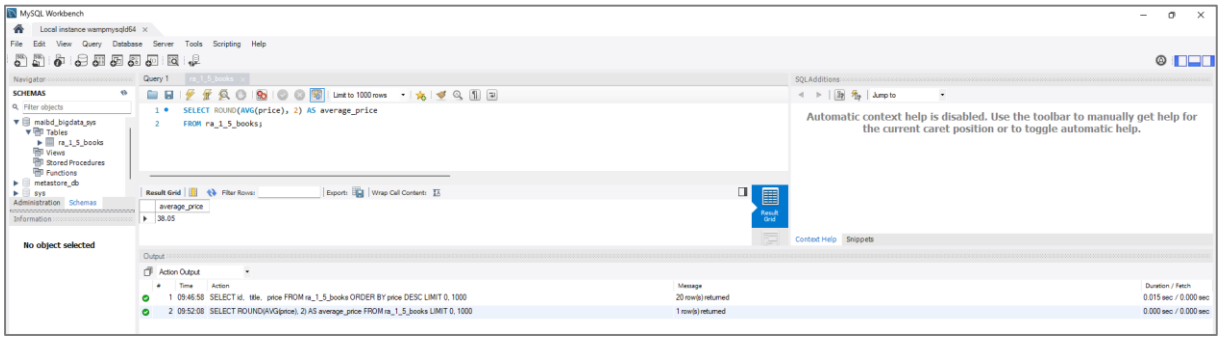


Fig. 6. MySQL Workbench

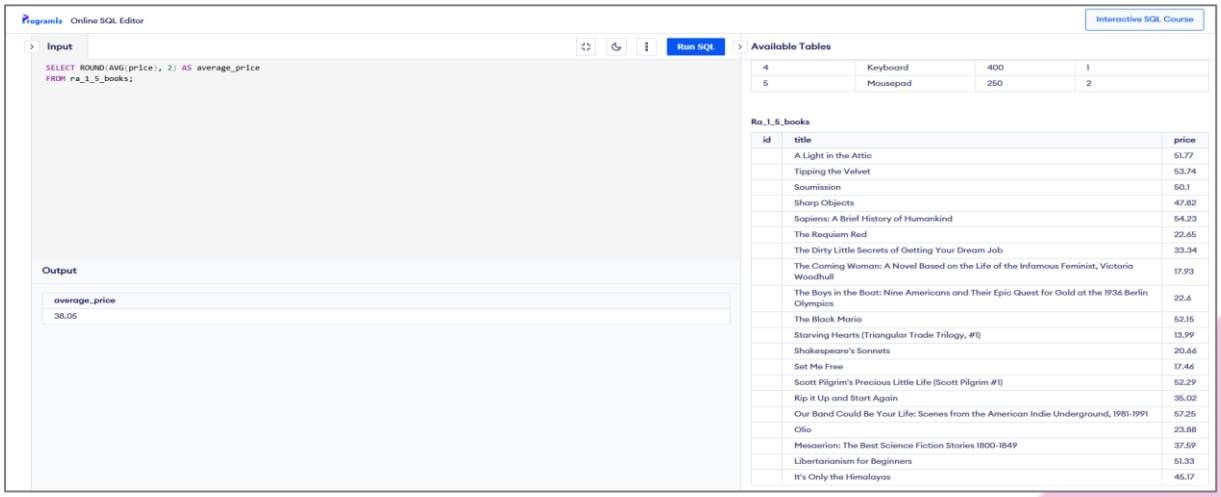


Fig. 7. Programiz

c. Encuentra el libro más caro y el más barato.

```sql
SELECT 'Most Expensive' AS category, id, title, price
FROM ra_1_5_books
WHERE price = (
    SELECT MAX(price)
    FROM ra_1_5_books
  )
UNION ALL
SELECT 'Cheapest' AS category, id, title, price
FROM ra_1_5_books
WHERE price = (
    SELECT MIN(price)
    FROM ra_1_5_books
  );
```

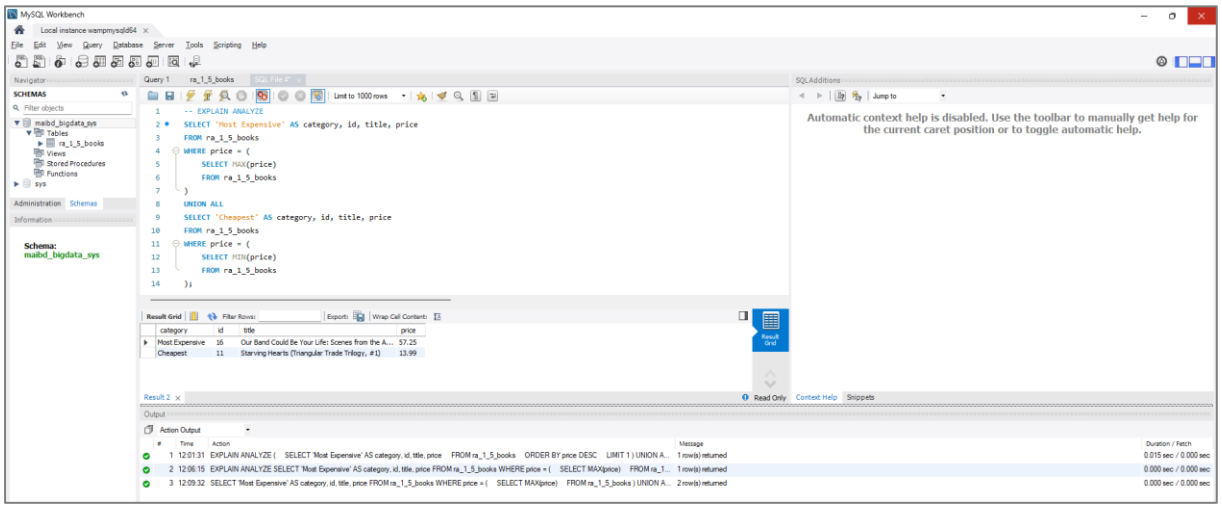


Fig. 6. MySQL Workbench

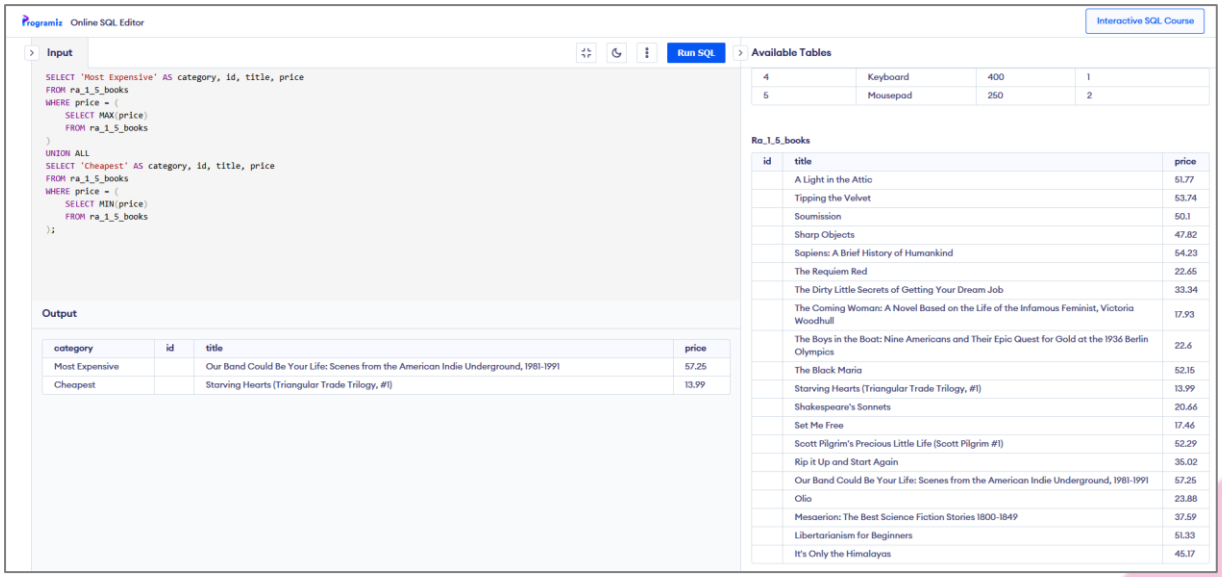


Fig. 7. Programiz

Aquí hay varias cuestiones importantes a analizar.

Una consulta `UNION [ALL]` casi nunca es la consulta más optimizada para ejecutar, sobre todo pensando en millones de registros porque normalmente hace recorridos completos de las tablas implicadas, todo esto dependiendo de la complejidad de las consultas, relaciones e índices.

En la consulta anterior no se tiene en cuenta la duplicidad del valor de la columna `price`, pudiendo resolverse con `LIMIT`.

Para ambas consultas se ha ejecutado un `ANALYZE` y nos acaba indicado, que el optimizador decide que, para una tabla de 20 filas, sale más barato leer `20 filas > ordenarlas > quedarse con 1`, que el índice por la columna `price`, es decir, que no está utilizando el índice creado `idx_ra_1_5_books_price` para resolver los `ORDER BY` de las consultas y no le voy a llevar la contraria ya que el coste real que muestra es ridículamente pequeño: `time=0.105 ms`.

```
-> Append  (cost=4.5 rows=2) (actual time=0.13..0.16 rows=2 loops=1)
  -> Stream results  (cost=2.25 rows=1) (actual time=0.128..0.129 rows=1 loops=1)
    -> Limit: 1 row(s)  (cost=2.25 rows=1) (actual time=0.105..0.106 rows=1 loops=1)
     -> Sort: ra_1_5_books.price DESC, limit input to 1 row(s) per chunk  (cost=2.25 rows=20) (actual
        time=0.105..0.105 rows=1 loops=1)
       -> Table scan on ra_1_5_books  (cost=2.25 rows=20) (actual time=0.0498..0.0646 rows=20 loops=1)
  -> Stream results  (cost=2.25 rows=1) (actual time=0.0294..0.0295 rows=1 loops=1)
    -> Limit: 1 row(s)  (cost=2.25 rows=1) (actual time=0.0281..0.0281 rows=1 loops=1)
     -> Sort: ra_1_5_books.price, limit input to 1 row(s) per chunk  (cost=2.25 rows=20) (actual
        time=0.0279..0.0279 rows=1 loops=1)
       -> Table scan on ra_1_5_books  (cost=2.25 rows=20) (actual time=0.0116..0.0187 rows=20 loops=1)
```

En resumen, por mucho que quiera optimizar la consulta, no tiene sentido hacerlo ya que no se puede sacar conclusiones de rendimiento con 20 registros.

d. Cuenta cuántos libros tienen un precio superior a 50.

```sql
SELECT COUNT(*) AS total_books_over_50
FROM ra_1_5_books
WHERE price > 50;
```

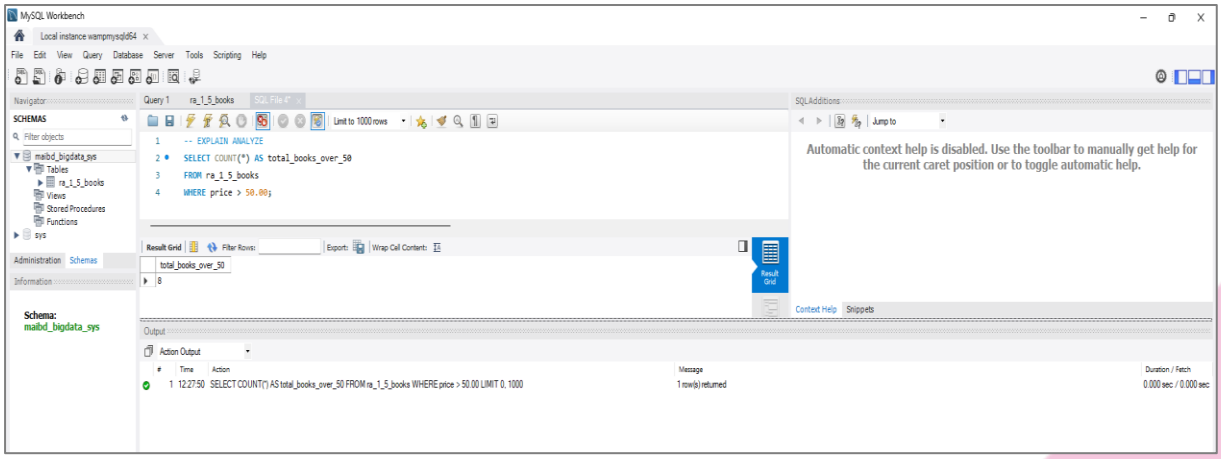


Fig. 6. MySQL Workbench

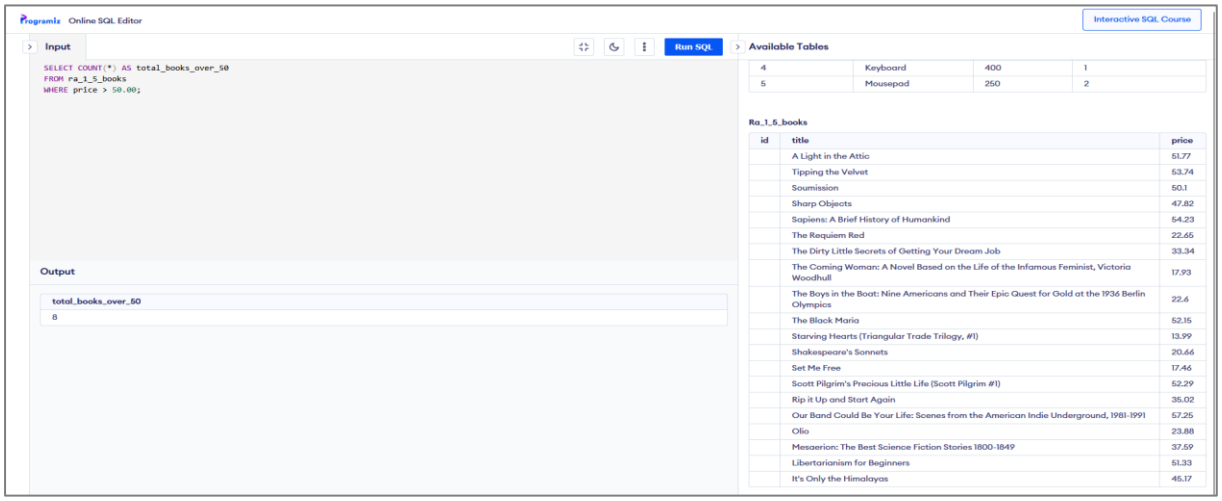


Fig. 7. Programiz

e. *Selecciona los títulos de los 5 libros más baratos.*

```sql
SELECT title, price
FROM ra_1_5_books
ORDER BY price DESC
LIMIT 5;
```

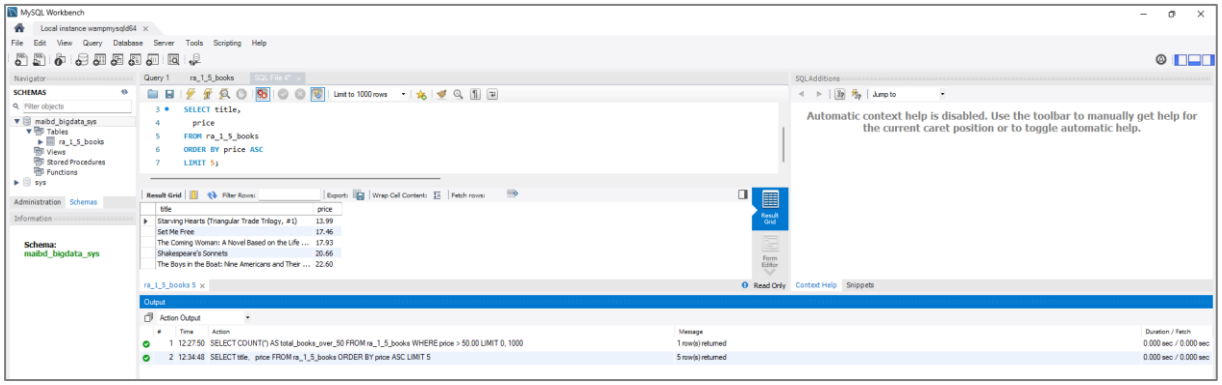


Fig. 6. MySQL Workbench

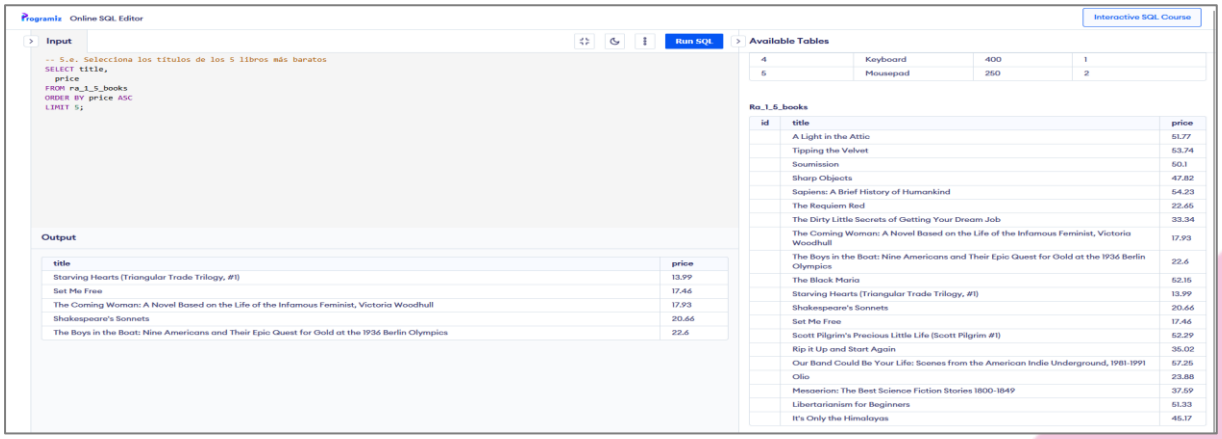


Fig. 7. Programiz

f. Calcula el precio total de todos los libros en la base de datos.

```
SELECT ROUND(MAX(price), 2) AS total_catalog_price
FROM ra_1_5_books;
```

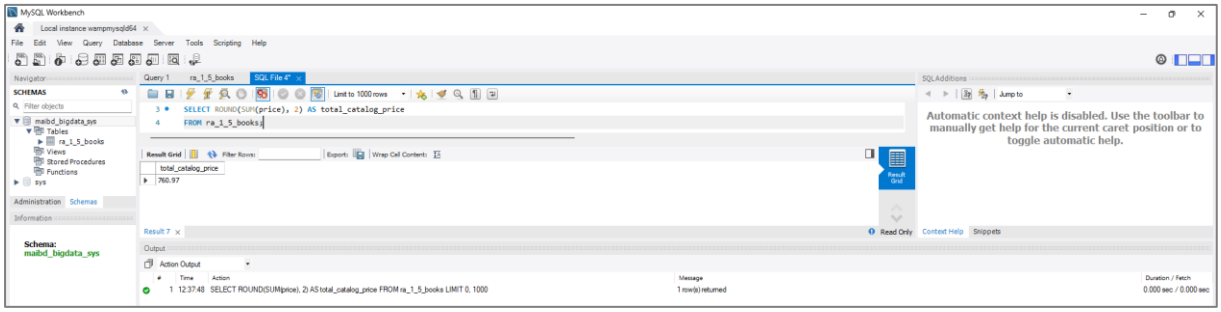


Fig. 6. MySQL Workbench

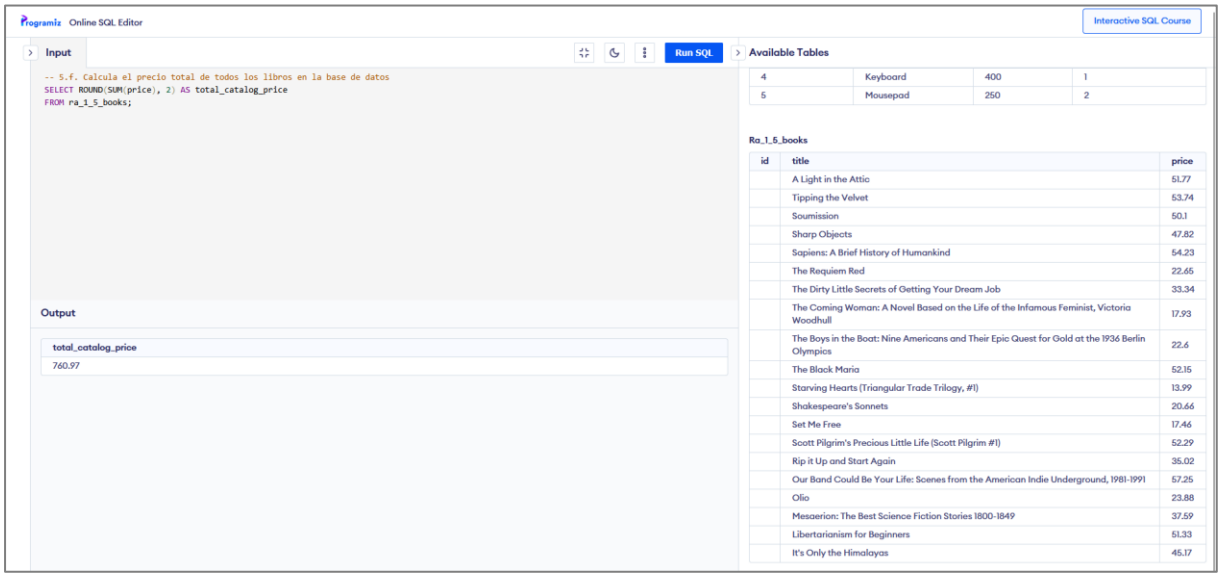


Fig. 7. Programiz

5. Consulta Avanzada

Crea una consulta que muestre el título del libro, su precio, y una columna adicional que indique si el libro es "Barato" (menor al promedio) o "Caro" (mayor o igual al promedio).

```
SELECT title, price,
  CASE
    WHEN price >= (
      SELECT AVG(price)
      FROM ra_1_5_books
    ) THEN 'Expensive'
    ELSE 'Cheap'
  END AS price_category
FROM ra_1_5_books
ORDER BY price DESC;
```

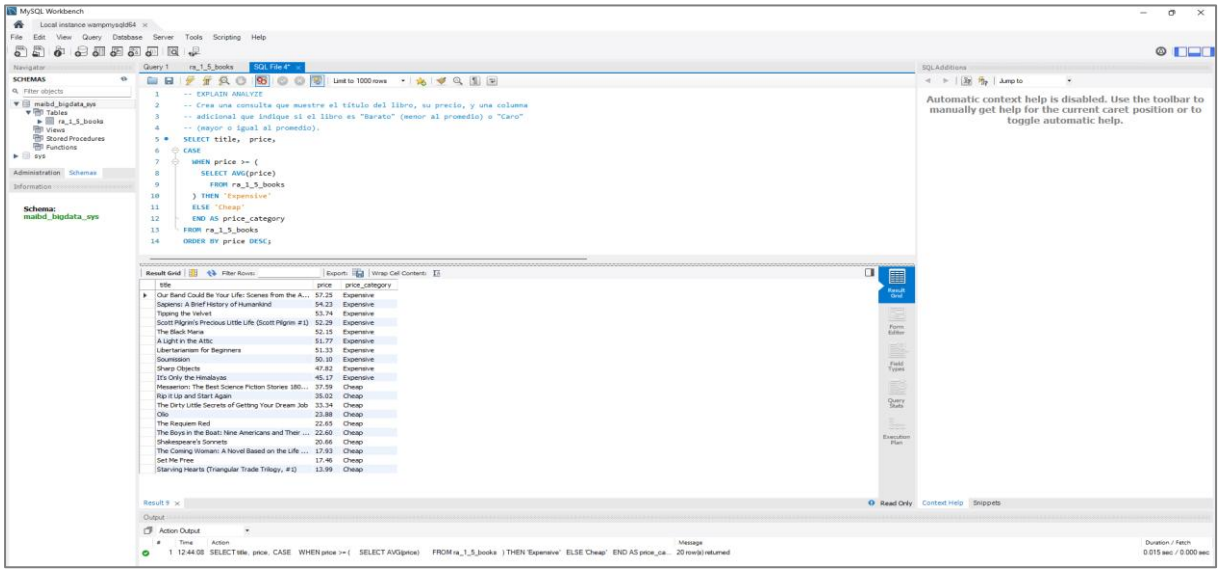


Fig. 6. MySQL Workbench

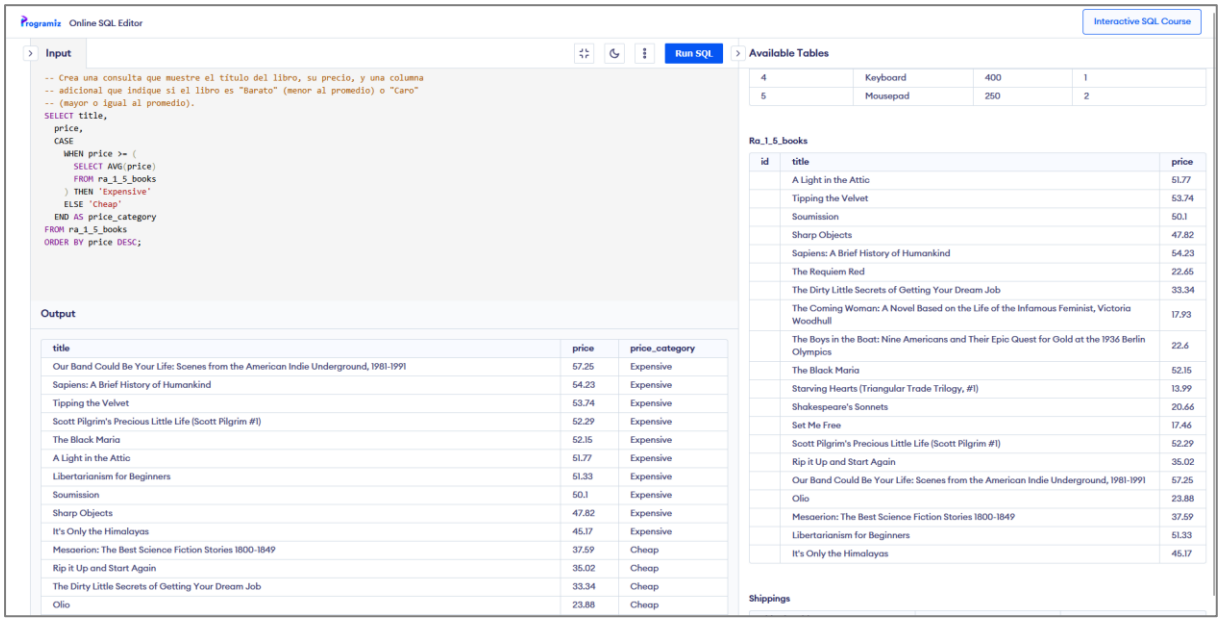


Fig. 7. Programiz

Aquí podemos sacar más provecho a los resultados de la consulta. Para añadir más valor, también podemos incluir el precio medio en el conjunto de resultados y categorizar cada libro como "Barato" o "Caro" según su precio relativo al promedio.

Esto proporciona un contexto más claro para el precio de cada libro y el por qué de “caro” o “barato”.

El objetivo aquí es mostrar un “catálogo” de libros ordenado por precio descendente e indicando si cada libro tiene un precio superior, inferior o igual al precio medio de las existencias. Además, se incorpora una fila con el precio medio calculado, para facilitar su consulta y servir como referencia para comparar el resto de los libros, para una posible exportación a hoja de cálculo.

```sql
-- [@dcueli optional] EXPLAIN ANALYZE

WITH book_stats AS (
  SELECT ROUND(AVG(price), 2) AS avg_price
  FROM ra_1_5_books
)
SELECT title, price, price_category
FROM (
  SELECT
    '' AS title,
    s.avg_price AS price,
    'Precio medio' AS price_category,
    0 AS sort_order
  FROM book_stats AS s
  UNION ALL
  SELECT b.title, b.price,
    CASE
      WHEN b.price = s.avg_price THEN 'Precio medio'
      WHEN b.price > s.avg_price THEN 'Caro (+' || CAST(
        ROUND((b.price - s.avg_price) / s.avg_price * 100, 2) AS CHAR(20)
      ) || '%)'
      ELSE 'Barato (-' || CAST(
        ROUND((s.avg_price - b.price) / s.avg_price * 100, 2) AS CHAR(20)
      ) || '%)'
    END AS price_category,
    1 AS sort_order
  FROM ra_1_5_books AS b
    CROSS JOIN book_stats AS s
) AS result
ORDER BY
  sort_order,
  price
DESC;
```

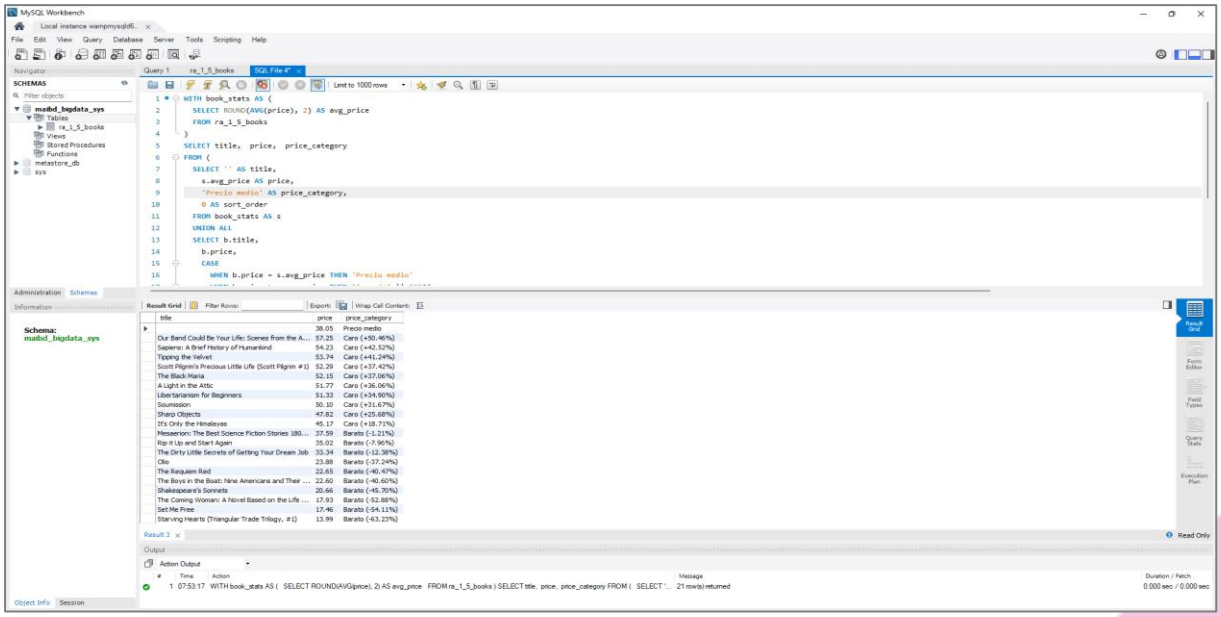


Fig. 6. MySQL Workbench

Haciendo un `EXPLAIN ANALYZE:`

```
> Sort: result.sort_order, result.price DESC  (cost=18.7..18.7 rows=21) (actual time=0.827..0.829 rows=21 loops=1)
  > Table scan on result  (cost=4.79..7.42 rows=21) (actual time=0.754..0.759 rows=21 loops=1)
    > Union all materialize  (cost=4.66..4.66 rows=21) (actual time=0.753..0.753 rows=21 loops=1)
      > Rows fetched before execution  (cost=0..0 rows=1) (actual time=100e-6..200e-6 rows=1 loops=1)
        > Table scan on b  (cost=2.56 rows=20) (actual time=0.0694..0.637 rows=20 loops=1)
```

Interpretando los datos podemos, o mejor dicho, puedo tener el siguiente resumen humanizado:

Operación	Información relevante
Table scan on b	Lee los 20 registros de la tabla de libros
Rows fetched before execution	Obtiene una fila con las estadísticas calculadas
Union all materialize	Combina y materializa las 21 filas del resultado: • 20 libros, • una fila informativa
Table scan on result	Recorre las 21 filas del resultado materializado
Sort	Ordena el resultado por sort_order y después por precio descendente

El tiempo total de la ordenación es de aproximadamente 0,829 ms en esta ejecución, pero sobre una tabla de 20 registros, esto no permite predecir directamente cuánto tardará la consulta con millones de registros, y a partir de aquí, hay un detalle bastante significativo y es que en este ejemplo, dicha *ordenación se realiza sobre el resultado completo y son solo 21 filas* (con la añadida)*, la ordenación sobre millones sería una operación mucho más costosa*.

Aquí, creo que es conveniente decir que añadir una fila con el precio medio no suma un problema de rendimiento, es decir, la adición de una fila más es insignificante frente al procesamiento de, me lo invento, 80 millones de libros.

En mi opinión, los costes importantes serían:

- ***Calcular la media***, donde la función `AVG(price)` debe considerar todos los precios. Sin una optimización que permita el uso de un índice, implica recorrer los datos. Un índice sobre el campo, o columna, `price` permite una lectura más estrecha y reducir la cantidad de datos leídos (cabe decir que ya lo incorporé en la definición de tabla*).*
- Al ***clasificar los libros*** hay que comparar cada precio con la media y si son 80 millones de comparaciones, aproximadamente será una por registro.
- ***Ordenar el resultado*** es una de las partes que más estresantes de la consulta, ya que hay que saber que son dos operaciones, la combinación de resultados y después la ordenación, por lo que, con 80 millones de filas, la ordenación puede requerir mucha memoria y recurrir al disco.
- ***Devolver los resultados*** al cliente también puede convertirse en un coste importante, independientemente de lo eficiente que sea la consulta.

En el caso hipotético de los 80 millones de registros, con 3 columnas en la tabla, se puede sumar también un caso real en el que no sean 3, sino 40 columnas (ya sabemos que en casos reales la normalización, en muchos casos, brilla por la ausencia) aunque esto puede o no multiplicar el coste ya que lo que realmente importa es qué columnas que se leen tienen índices.